An Annotated Reconstruction of Zhao–Cui Tensor-Train Sequential Filtering

BayesFilter Technical Note

2026-06-01

Abstract

This note reconstructs the tensor-train sequential filtering method of Zhao and Cui in BayesFilter notation. The goal is not to compress the method, but to slow it down until the main operations can be followed by a reader who knows Bayesian state-space models and calculus but has not studied tensor trains. The first part follows the paper's state-space model, recursive posterior formula, basic TT approximation, squared-TT approximation, conditional Knothe–Rosenblatt maps, particle correction, backward smoothing, and preconditioning ideas. The second part isolates a fixed-branch variant needed for a same-scalar analytical derivative. Two propositions state precisely what is proved: the fixed-branch recursion is a normalized approximate filtering recursion for its own declared approximation, and the fixed-branch derivative differentiates the same approximate likelihood scalar computed by the forward pass.

Contents

1 What Problem Is Being Solved?

The data arrive in time order. At time t there is a hidden state

$$x_t \in \mathbb{R}^{m_x}$$

and an observation

$$y_t \in \mathbb{R}^{m_y}.$$

There may also be a static unknown parameter

$$\alpha \in \mathbb{R}^{m_\alpha}.$$

The filtering problem is to update the conditional density of the current state and parameter when y_t arrives:

$$p(x_t, \alpha \mid y_{1:t}).$$

The path problem is to update the density of the whole trajectory and parameter:

$$p(x_{0:t}, \alpha \mid y_{1:t}).$$

The smoothing problem asks for an earlier state after later data have been seen:

$$p(x_k \mid y_{1:t}), \qquad k < t.$$

The bottleneck is always an integral. At time $t-1$ suppose we know, or have approximated,

$$p(x_{t-1}, \alpha \mid y_{1:t-1}).$$

After seeing y_t, Bayes' rule introduces the unnormalized three-block density

$$q_t(x_t, \alpha, x_{t-1}) = p(x_{t-1}, \alpha \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

The next filter is obtained by integrating out x_{t-1}:

$$p(x_t, \alpha \mid y_{1:t}) = \frac{\int q_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_{t-1}}{\int q_t(x_t, \alpha, x_{t-1})\, \mathrm{d}x_t\, \mathrm{d}\alpha\, \mathrm{d}x_{t-1}}.$$

In low dimension one can attack this integral by quadrature, particles, or Gaussian projection. Zhao–Cui's idea is different: approximate the whole function q_t in a separable tensor-train form so that marginal integrals are cheap contractions instead of high-dimensional numerical integrations.

2 State-Space Model And BayesFilter Notation

Zhao–Cui use θ for the unknown parameter. In this note the parameter is written α, because θ is often used later as a generic argument in derivative formulas. The paper's model equations (1)–(2) become

$$X_t \mid (X_{0:t-1} = x_{0:t-1}, \alpha) \equiv X_t \mid (X_{t-1} = x_{t-1}, \alpha) \sim f_\alpha(x_t \mid x_{t-1}), \tag{BF-1}$$

and

$$Y_t \mid (X_{0:t} = x_{0:t}, Y_{1:t-1} = y_{1:t-1}, \alpha) \equiv Y_t \mid (X_t = x_t, \alpha) \sim g_\alpha(y_t \mid x_t). \tag{BF-2}$$

The first statement says the latent state process is Markov once α is fixed. The second says the new observation only sees the current state, again once α is fixed.

The joint density of parameter, states, and observations through time t, corresponding to Zhao–Cui Eq. (3), is

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha) p_\alpha(x_0) \prod_{j=1}^{t} f_\alpha(x_j \mid x_{j-1}) g_\alpha(y_j \mid x_j). \tag{BF-3}$$

This is obtained by the chain rule:

$$\begin{aligned} p(\alpha, x_{0:t}, y_{1:t}) &= p(\alpha) p_\alpha(x_0) \prod_{j=1}^{t} p(x_j \mid x_{0:j-1}, y_{1:j-1}, \alpha) p(y_j \mid x_{0:j}, y_{1:j-1}, \alpha) \\ &= p(\alpha) p_\alpha(x_0) \prod_{j=1}^{t} f_\alpha(x_j \mid x_{j-1}) g_\alpha(y_j \mid x_j), \end{aligned}$$

where the last equality uses the Markov and observation assumptions.

Conditioning on data gives Zhao–Cui Eq. (4) in BayesFilter notation:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}, \qquad p(y_{1:t}) = \int p(\alpha, x_{0:t}, y_{1:t}) \, \mathrm{d}\alpha \, \mathrm{d}x_{0:t}. \tag{BF-4}$$

The denominator is the evidence. It is a scalar, but it is usually not available analytically because the integral spans the parameter and all states.

3 The Four Marginal Learning Problems

Zhao–Cui Eqs. (5)–(8) are all marginalizations of the same posterior $p(\alpha, x_{0:t} \mid y_{1:t})$. Filtering keeps only the current state:

$$p(x_t \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t-1} \, \mathrm{d}\alpha. \tag{BF-5}$$

Parameter learning keeps only α:

$$p(\alpha \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t}. \tag{BF-6}$$

Path learning keeps the full state path but removes α:

$$p(x_{0:t} \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}\alpha. \tag{BF-7}$$

Fixed-lag or retrospective smoothing keeps one old state:

$$p(x_k \mid y_{1:t}) = \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:k-1} \, \mathrm{d}x_{k+1:t} \, \mathrm{d}\alpha, \qquad k < t. \tag{BF-8}$$

The important point is not the names. It is that the same high-dimensional posterior density produces all four outputs by integration. A method that stores only a cloud of samples can estimate these integrals by sample averages. A method that stores a Gaussian can compute only Gaussian marginals. A method that stores a tensor-train density aims to keep a non-Gaussian function representation while making many marginal integrals cheap.

4 The Exact Recursive Bottleneck

We now derive Zhao–Cui Eqs. (9)–(11). Start from Bayes' rule:

$$p(\alpha, x_{0:t} \mid y_{1:t}) = \frac{p(\alpha, x_{0:t}, y_{1:t})}{p(y_{1:t})}.$$

Using the factorization (BF-3),

$$p(\alpha, x_{0:t}, y_{1:t}) = p(\alpha, x_{0:t-1}, y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Also,

$$p(\alpha, x_{0:t-1}, y_{1:t-1}) = p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}).$$

Therefore

$$\begin{aligned} p(\alpha, x_{0:t} \mid y_{1:t}) &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) p(y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_{1:t})} \\ &= \frac{p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \tag{BF-9}$$

The last step uses $p(y_{1:t}) = p(y_t \mid y_{1:t-1}) p(y_{1:t-1})$.

Now integrate (BF-9) over $x_{0:t-2}$. The only factor depending on the old path $x_{0:t-2}$ is $p(\alpha, x_{0:t-1} \mid y_{1:t-1})$, so

$$\begin{aligned} p(\alpha, x_t, x_{t-1} \mid y_{1:t}) &= \int p(\alpha, x_{0:t} \mid y_{1:t}) \, \mathrm{d}x_{0:t-2} \\ &= \frac{p(\alpha, x_{t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{p(y_t \mid y_{1:t-1})}. \end{aligned} \tag{BF-10}$$

Finally, integrate over x_{t-1} to obtain the next parameter-state filter:

$$p(\alpha, x_t \mid y_{1:t}) = \int p(\alpha, x_t, x_{t-1} \mid y_{1:t}) \, \mathrm{d}x_{t-1}. \tag{BF-11}$$

The numerator in (BF-10) is pointwise evaluable if the previous filter is evaluable. The denominator and the marginalization in (BF-11) are the hard parts. Zhao–Cui turn this into a function-approximation problem by replacing the unnormalized numerator with a tensor train.

5 Tensor Trains From First Principles

This section starts from the object a numerical reader can picture: a table of function values. If $D = 2$, a function $h(r_1, r_2)$ sampled on a grid is a matrix. Low-rank matrix approximation writes

$$h(r_1, r_2) \approx \sum_{\rho=1}^{R} u_\rho(r_1) v_\rho(r_2).$$

Each term is separable: one factor depends only on r_1, one factor only on r_2. A tensor train is the many-variable analogue. It does not require a full D-dimensional table. It stores a chain of low-rank links, so that information can pass from coordinate 1 to coordinate D through the rank indices.

Let $r = (r_1, \ldots, r_D) \in \Omega_1 \times \cdots \times \Omega_D$. A functional tensor train approximates a scalar function $h(r)$ by multiplying matrix-valued one-dimensional functions:

$$h(r) \approx \widehat{h}(r) = H_1(r_1) H_2(r_2) \cdots H_D(r_D), \tag{TT-1}$$

where

$$H_k(r_k) \in \mathbb{R}^{R_{k-1} \times R_k}, \qquad R_0 = R_D = 1.$$

Because the first object has one row and the last has one column, the product is a 1×1 scalar. Written with indices,

$$\widehat{h}(r) = \sum_{\rho_1=1}^{R_1} \cdots \sum_{\rho_{D-1}=1}^{R_{D-1}} H_1^{1,\rho_1}(r_1) H_2^{\rho_1,\rho_2}(r_2) \cdots H_D^{\rho_{D-1},1}(r_D). \tag{TT-2}$$

The numbers R_k are TT ranks. A high rank lets the representation express strong interactions across a split $(r_1, \ldots, r_k)$ and $(r_{k+1}, \ldots, r_D)$. A low rank is useful only when the function has low-dimensional structure across that split.

To see the role of the ranks, split the variables into a left block and a right block:

$$r_{\le k} = (r_1, \dots, r_k), \qquad r_{>k} = (r_{k+1}, \dots, r_D).$$

Equation (TT-2) can be regrouped as

$$\widehat{h}(r_{\le k}, r_{>k}) = \sum_{\rho=1}^{R_k} L_\rho(r_{\le k}) R_\rho(r_{>k}), \tag{TT-2a}$$

where

$$L_\rho(r_{\le k}) = \sum_{\rho_1, \dots, \rho_{k-1}} H_1^{1,\rho_1}(r_1) \cdots H_k^{\rho_{k-1},\rho}(r_k),$$

and

$$R_\rho(r_{>k}) = \sum_{\rho_{k+1}, \dots, \rho_{D-1}} H_{k+1}^{\rho,\rho_{k+1}}(r_{k+1}) \cdots H_D^{\rho_{D-1},1}(r_D).$$

Thus R_k is the number of separated terms allowed across that split. If a posterior density has only local or weak long-range dependence in the chosen ordering, moderate ranks can be enough. If a posterior has strong global coupling, the ranks grow and the method becomes expensive.

Each univariate matrix entry is represented in a basis:

$$H_k^{a,b}(r_k) = \sum_{\ell=0}^{p_k-1} C_k[a, \ell, b] \psi_{k,\ell}(r_k). \tag{TT-3}$$

The tensor $C_k \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}$ is the k-th TT core. Thus an implementation stores the basis family, the domain map for each coordinate, and the core arrays $C_1, \dots, C_D$. To evaluate the function at one point r, it evaluates all basis vectors

$$\psi_k(r_k) = (\psi_{k,0}(r_k), \dots, \psi_{k,p_k-1}(r_k))^\top,$$

forms the matrices $H_k(r_k)$, and multiplies them.

5.1 A Concrete Basis Choice

Zhao–Cui's software supports several basis families. For a self-contained minimal implementation, choose one. On a finite interval $[a_k, b_k]$, map a physical coordinate r_k to

$$z_k = \frac{2r_k - (a_k + b_k)}{b_k - a_k} \in [-1, 1], \qquad r_k = \frac{a_k + b_k}{2} + \frac{b_k - a_k}{2} z_k. \tag{TT-6}$$

Use normalized Legendre polynomials

$$\psi_{k,\ell}(r_k) = \sqrt{\frac{2\ell + 1}{b_k - a_k}}\, P_\ell(z_k), \qquad \ell = 0, \dots, p_k - 1. \tag{TT-7}$$

The Legendre polynomials are evaluated by the recurrence

$$P_0(z) = 1, \qquad P_1(z) = z,$$
$$(\ell + 1) P_{\ell+1}(z) = (2\ell + 1) z P_\ell(z) - \ell P_{\ell-1}(z), \qquad \ell \ge 1. \tag{TT-8}$$

This basis is useful pedagogically because its mass matrix is the identity:

$$\int_{a_k}^{b_k} \psi_{k,\ell}(r_k) \psi_{k,\ell'}(r_k)\, \mathrm{d}r_k = \delta_{\ell\ell'}. \tag{TT-9}$$

If another basis is used, every formula below remains the same after replacing the identity mass matrix by the corresponding one-dimensional mass matrix.

6 Why TT Marginalization Is Cheap Once The Representation Exists

Suppose h is represented by (TT-1). If we need to integrate out coordinate r_k, the product structure gives

$$\int \widehat{h}(r_1, \dots, r_D)\, \mathrm{d}r_k = H_1(r_1) \cdots H_{k-1}(r_{k-1}) \left(\int H_k(r_k)\, \mathrm{d}r_k \right) H_{k+1}(r_{k+1}) \cdots H_D(r_D). \tag{TT-4}$$

Only the k-th core changes. Its integrated matrix is

$$\overline{H}_k = \int H_k(r_k)\, \mathrm{d}r_k, \qquad \overline{H}_k[a, b] = \sum_{\ell=0}^{p_k - 1} C_k[a, \ell, b] \int \psi_{k,\ell}(r_k)\, \mathrm{d}r_k. \tag{TT-5}$$

If the one-dimensional basis integrals are precomputed, marginalizing one coordinate is a small matrix contraction. Repeatedly applying (TT-4) integrates whole blocks of variables.

This explains why Zhao–Cui target a TT representation of (x_t, α, x_{t-1}). Once that three-block function is separable enough, the new filter $p(x_t, \alpha \mid y_{1:t})$ is obtained by integrating the old-state block x_{t-1}, one coordinate at a time.

7 Zhao–Cui Algorithm 1, Fully Annotated

The exact recursion (BF-10) contains the previous exact filter. A numerical algorithm has the previous approximation. Let $\widehat{\pi}_{t-1}(x_{t-1}, \alpha)$ denote an unnormalized approximation of $p(x_{t-1}, \alpha \mid y_{1:t-1})$. Zhao–Cui Eq. (12) becomes

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{A1-1}$$

This is not separable in general. Even if $\widehat{\pi}_{t-1}$ is a TT over (α, x_{t-1}), the transition and likelihood couple the new and old state coordinates.

Algorithm 1 has three mathematical steps.

Step 1: build a pointwise target. Given $r_t = (x_t, \alpha, x_{t-1})$, evaluate $q_t(r_t)$ by multiplying the previous approximation, transition density, and likelihood:

$$r_t \mapsto \widehat{\pi}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Storage needed: an evaluator for the previous TT approximation and evaluators for f_α and g_α.

Step 2: re-approximate by a TT. Choose a variable order, in the paper usually

$$(x_t, \alpha, x_{t-1}).$$

Fit a TT

$$\widehat{\pi}_t(x_t, \alpha, x_{t-1}) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) H_{t,1}(x_{t-1,1}) \cdots H_{t,m_x}(x_{t-1,m_x}) \tag{A1-2}$$

so that $\widehat{\pi}_t \approx q_t$. Zhao–Cui use functional TT tools with alternating approximation and rank adaptation. For implementation one must choose a domain map, basis, fitting points, rank rule,

and stopping rule. The paper's research code supplies one such family; the fixed-branch section below supplies a differentiable BayesFilter specialization.

The most direct implementation view is least squares. Choose fitting points

$$r^{(j)} = (x_t^{(j)}, \alpha^{(j)}, x_{t-1}^{(j)}), \qquad j = 1, \dots, N_{\text{fit}},$$

and define target values

$$y_j = q_t(r^{(j)}).$$

For fixed TT ranks and fixed all cores except core k, the fitted function is linear in the entries of core C_k. There is a design row $A_{j,k}$ such that

$$\widehat{\pi}_t(r^{(j)}) = A_{j,k} g_k, \qquad g_k = \operatorname{vec}(C_k). \tag{A1-2a}$$

Let $L_{j,k}$ be the left environment obtained by multiplying all cores before k at point $r^{(j)}$, and let $R_{j,k}$ be the right environment from all cores after k. Let $\psi_k(r_k^{(j)})$ be the local basis vector. Then

$$A_{j,k} = R_{j,k}^\top \otimes \psi_k(r_k^{(j)})^\top \otimes L_{j,k}. \tag{A1-2b}$$

This formula is worth lingering over. The left environment tells the current core what the earlier coordinates have already contributed. The right environment tells it what later coordinates will contribute. The basis vector gives the local coordinate dependence. The Kronecker product stitches these three pieces into a row that multiplies the vectorized core.

With weights $w_j > 0$, the core update solves

$$\min_{g_k} \sum_{j=1}^{N_{\text{fit}}} w_j \big(A_{j,k} g_k - y_j\big)^2 + \rho \|g_k\|_2^2, \tag{A1-2c}$$

or equivalently

$$(A_k^\top W A_k + \rho I) g_k = A_k^\top W y. \tag{A1-2d}$$

An alternating scheme sweeps through the cores, updating one core at a time. Zhao–Cui's adaptive implementation is more sophisticated, but this least- squares view is the easiest way to see how a TT approximation becomes an actual numerical object.

Step 3: integrate the old state. The next approximate filter is

$$\widehat{\pi}_t(x_t, \alpha) = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_{t-1}. \tag{A1-3}$$

Because x_{t-1} occupies the last block, this integral is a sequence of core integrals:

$$\widehat{\pi}_t(x_t, \alpha) = F_{t,1}(x_{t,1}) \cdots F_{t,m_x}(x_{t,m_x}) A_{t,1}(\alpha_1) \cdots A_{t,m_\alpha}(\alpha_{m_\alpha}) \overline{H}_{t,1} \cdots \overline{H}_{t,m_x}. \tag{A1-4}$$

The approximate normalizer is

$$\widehat{Z}_t = \int \widehat{\pi}_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_t \, \mathrm{d}\alpha \, \mathrm{d}x_{t-1}, \tag{A1-5}$$

again a complete TT contraction.

The main failure mode of Algorithm 1 is sign. A fitted TT is a real-valued function approximation. Truncation or interpolation can make $\widehat{\pi}_t(r_t) < 0$, which is impossible for a density. Negative values also break transport maps and importance-sampling ratios. This motivates the squared-TT construction.

8 Why Nonnegativity Fails And Why Square-Root TT Repairs It

Approximating a nonnegative function by a truncated expansion does not preserve nonnegativity. The one-dimensional analogy is simple:

$$h(x) \geq 0 \quad \text{but} \quad \widehat{h}(x) = \sum_{\ell=0}^{p-1} c_\ell \psi_\ell(x) \text{ may be negative.}$$

The tensor-train case has the same issue. A low-rank approximation can reduce error in a least-squares sense while still creating small negative regions.

Zhao–Cui's repair is to approximate the square root of an unnormalized density. If

$$\sqrt{\pi(r)} \approx \phi(r),$$

then

$$\phi(r)^2 \geq 0$$

for all r. A small defensive density is then added so that the proposal density is positive wherever the target might be positive.

9 Squared-TT Density, Defensive Reference, And Normalizer

Zhao–Cui Eq. (13) becomes the following. Let $\pi(r) \geq 0$ be an unnormalized target on a domain $\Omega \subseteq \mathbb{R}^D$, with normalizer

$$Z = \int_\Omega \pi(r)\,\mathrm{d}r.$$

Let $\lambda(r)$ be a normalized reference density on Ω,

$$\lambda(r) \geq 0, \qquad \int_\Omega \lambda(r)\,\mathrm{d}r = 1,$$

and choose $\tau > 0$. Fit a TT $\phi(r)$ to $\sqrt{\pi(r)}$. Define

$$\widehat{\pi}(r) = \phi(r)^2 + \tau\lambda(r), \qquad \widehat{Z} = \int_\Omega \widehat{\pi}(r)\,\mathrm{d}r, \qquad \widehat{p}(r) = \frac{\widehat{\pi}(r)}{\widehat{Z}}. \tag{S1}$$

Since λ is normalized,

$$\widehat{Z} = \int_\Omega \phi(r)^2\,\mathrm{d}r + \tau. \tag{S2}$$

The defensive term has two jobs. First, it keeps the approximation positive even if ϕ is zero somewhere. Second, if π/λ is bounded, then

$$\frac{p(r)}{\widehat{p}(r)} = \frac{\pi(r)/Z}{\widehat{\pi}(r)/\widehat{Z}} = \frac{\widehat{Z}}{Z}\frac{\pi(r)}{\phi(r)^2 + \tau\lambda(r)} \leq \frac{\widehat{Z}}{Z\tau}\frac{\pi(r)}{\lambda(r)}. \tag{S3}$$

This is the importance-sampling reason for the defensive density: the true target is absolutely continuous with respect to the approximation when the reference covers the target support.

10 Squared-TT Marginalization And Mass Matrices

The square changes the marginalization algebra. Let

$$\phi(r) = H_1(r_1)\cdots H_D(r_D), \qquad H_k(r_k) \in \mathbb{R}^{R_{k-1}\times R_k}.$$

For a split after coordinate k, define

$$H_{\le k}(r_{\le k}) = H_1(r_1)\cdots H_k(r_k) \in \mathbb{R}^{1\times R_k},$$

and

$$H_{>k}(r_{>k}) = H_{k+1}(r_{k+1})\cdots H_D(r_D) \in \mathbb{R}^{R_k\times 1}.$$

Then

$$\phi(r) = H_{\le k}(r_{\le k})H_{>k}(r_{>k}).$$

The marginal square term is

$$\begin{aligned}\int \phi(r)^2\,\mathrm{d}r_{>k} &= \int \left[H_{\le k}(r_{\le k})H_{>k}(r_{>k})\right]^2\,\mathrm{d}r_{>k} \\ &= \sum_{a=1}^{R_k}\sum_{b=1}^{R_k} H_{\le k,a}(r_{\le k})\left[\int H_{>k,a}(r_{>k})H_{>k,b}(r_{>k})\,\mathrm{d}r_{>k}\right] H_{\le k,b}(r_{\le k}).\end{aligned} \tag{M1}$$

Define the right mass matrix

$$M_{>k}[a,b] = \int H_{>k,a}(r_{>k})H_{>k,b}(r_{>k})\,\mathrm{d}r_{>k}. \tag{M2}$$

This matrix is positive semidefinite. If $M_{>k} = L_{>k}L_{>k}^\top$ is a Cholesky or square-root factorization, then

$$\int \phi(r)^2\,\mathrm{d}r_{>k} = \sum_{\gamma=1}^{R_k}\left[\sum_{a=1}^{R_k} H_{\le k,a}(r_{\le k})L_{>k}[a,\gamma]\right]^2. \tag{M3}$$

Including the defensive reference, if $\lambda(r) = \prod_{j=1}^D \lambda_j(r_j)$, gives

$$\widehat{p}(r_{\le k}) = \frac{\sum_{\gamma=1}^{R_k}\left[\sum_{a=1}^{R_k} H_{\le k,a}(r_{\le k})L_{>k}[a,\gamma]\right]^2 + \tau\prod_{j=1}^k \lambda_j(r_j)}{\widehat{Z}}. \tag{M4}$$

This is Zhao–Cui Eq. (14) in BayesFilter notation.

The practical recursion starts from the right end. Set $M_{>D} = 1$. Moving from $j = D$ down to 1, compute

$$M_{>j-1} = \int H_j(r_j)M_{>j}H_j(r_j)^\top\,\mathrm{d}r_j. \tag{M5}$$

In basis coefficients this integral is finite-dimensional. If

$$H_j^{a,b}(r_j) = \sum_\ell C_j[a,\ell,b]\psi_{j,\ell}(r_j),$$

and

$$B_j[\ell,\ell'] = \int \psi_{j,\ell}(r_j)\psi_{j,\ell'}(r_j)\,\mathrm{d}r_j,$$

then

$$M_{>j-1}[a,a'] = \sum_{b,b'=1}^{R_j}\sum_{\ell,\ell'} C_j[a,\ell,b]\,M_{>j}[b,b']\,C_j[a',\ell',b']\,B_j[\ell,\ell']. \tag{M6}$$

This formula is the bridge from the paper to code: marginalization stores mass matrices or their square roots and never enumerates a full tensor grid.

10.1 A Small Three-Coordinate Marginalization

Take $D = 3$, and suppose

$$\phi(r_1, r_2, r_3) = H_1(r_1)H_2(r_2)H_3(r_3).$$

If $H_1 \in \mathbb{R}^{1\times R_1}$, $H_2 \in \mathbb{R}^{R_1\times R_2}$, and $H_3 \in \mathbb{R}^{R_2\times 1}$, then marginalizing out r_3 gives

$$\int \phi(r_1, r_2, r_3)^2 \,\mathrm{d}r_3 = H_1(r_1)H_2(r_2)\left[\int H_3(r_3)H_3(r_3)^\top \,\mathrm{d}r_3\right] H_2(r_2)^\top H_1(r_1)^\top. \tag{M7}$$

The bracketed quantity is an $R_2 \times R_2$ matrix. It is not a function of r_1 or r_2; it is a stored contraction. If we also marginalize out r_2, then

$$M_{>1} = \int H_2(r_2)\left[\int H_3(r_3)H_3(r_3)^\top \,\mathrm{d}r_3\right] H_2(r_2)^\top \,\mathrm{d}r_2, \tag{M8}$$

and the one-coordinate marginal is

$$\int \phi(r_1, r_2, r_3)^2 \,\mathrm{d}r_2\,\mathrm{d}r_3 = H_1(r_1)M_{>1}H_1(r_1)^\top. \tag{M9}$$

This is exactly what the recursive mass formula (M5) does. The notation in the paper is compact because it assumes the reader already sees this matrix chain. The implementation sees it as repeated multiplication and integration of small matrices.

11 Conditional Densities And KR Maps From Marginal Ratios

For a normalized density $\widehat{p}(r_1, \ldots, r_D)$, the chain rule gives

$$\widehat{p}(r) = \widehat{p}(r_1)\prod_{k=2}^{D} \widehat{p}(r_k \mid r_{<k}), \qquad \widehat{p}(r_k \mid r_{<k}) = \frac{\widehat{p}(r_{\le k})}{\widehat{p}(r_{<k})}. \tag{K1}$$

The squared-TT marginal formula (M4) supplies both numerator and denominator. Thus each conditional density is computable from TT contractions.

Define conditional distribution functions

$$F_1(r_1) = \int_{-\infty}^{r_1} \widehat{p}(s_1)\,\mathrm{d}s_1, \tag{K2}$$

and, for $k \ge 2$,

$$F_k(r_k \mid r_{<k}) = \int_{-\infty}^{r_k} \widehat{p}(s_k \mid r_{<k})\,\mathrm{d}s_k. \tag{K3}$$

The lower triangular Knothe–Rosenblatt map is

$$F(r) = \big(F_1(r_1), F_2(r_2 \mid r_1), \ldots, F_D(r_D \mid r_{<D})\big). \tag{K4}$$

If $R \sim \widehat{p}$, then $F(R)$ is uniform on $[0,1]^D$. Conversely, if $\Xi \sim \mathrm{Unif}([0,1]^D)$, then $F^{-1}(\Xi) \sim \widehat{p}$.

The proof is the triangular Jacobian calculation. The Jacobian of F is lower triangular, and

$$\frac{\partial F_k(r_k \mid r_{<k})}{\partial r_k} = \widehat{p}(r_k \mid r_{<k}).$$

Therefore

$$|\det \nabla F(r)| = \prod_{k=1}^{D} \widehat{p}(r_k \mid r_{<k}) = \widehat{p}(r). \tag{K5}$$

The pullback of the uniform density by F is exactly $|\det \nabla F(r)| = \widehat{p}(r)$.

12 Zhao–Cui Algorithm 2, Fully Annotated

Algorithm 2 is Algorithm 1 with a square-root TT and a defensive density. At time $t-1$, assume the previous filter approximation is evaluable as

$$\widehat{p}_{t-1}(x_{t-1}, \alpha).$$

The pointwise unnormalized target, Zhao–Cui Eq. (15), is

$$q_t(x_t, \alpha, x_{t-1}) = \widehat{p}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{A2-1}$$

Fit a TT to the square root:

$$\sqrt{q_t(x_t, \alpha, x_{t-1})} \approx \phi_t(x_t, \alpha, x_{t-1}). \tag{A2-2}$$

Then define Zhao–Cui Eq. (16):

$$\widehat{q}_t(x_t, \alpha, x_{t-1}) = \phi_t(x_t, \alpha, x_{t-1})^2 + \tau_t \lambda_t(x_t) \lambda_\alpha(\alpha) \lambda_{t-1}(x_{t-1}). \tag{A2-3}$$

The approximate normalizer is

$$\widehat{Z}_t = \int \widehat{q}_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_t \, \mathrm{d}\alpha \, \mathrm{d}x_{t-1}. \tag{A2-4}$$

The next normalized approximate filter is

$$\widehat{p}_t(x_t, \alpha) = \frac{\int \widehat{q}_t(x_t, \alpha, x_{t-1}) \, \mathrm{d}x_{t-1}}{\widehat{Z}_t}. \tag{A2-5}$$

Everything is now nonnegative. The approximation risk has moved: the method must fit the square root accurately enough, choose a defensive term that is not too small for stability or too large for accuracy, and keep TT ranks manageable.

13 Forward Conditional Map And Particle-Filter Correction

The same $\widehat{q}_t(x_t, \alpha, x_{t-1})$ defines a conditional density for the new state given the old state and parameter:

$$\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) = \frac{\widehat{q}_t(x_t, \alpha, x_{t-1})}{\int \widehat{q}_t(s, \alpha, x_{t-1}) \, \mathrm{d}s}. \tag{F1}$$

Constructing the corresponding upper triangular conditional KR map gives a proposal sampler:

$$\widehat{X}_t^{(i)} = (F_{t,t}^u)^{-1} \left(\Xi_t^{(i)} \mid \widehat{\alpha}^{(i)}, \widehat{x}_{t-1}^{(i)} \right), \qquad \Xi_t^{(i)} \sim \mathrm{Unif}([0,1]^{m_x}). \tag{F2}$$

This is Zhao–Cui Eq. (21) in notation adapted to BayesFilter.

The proposal density for the full path after sampling is

$$\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t}) p(\alpha, x_{0:t-1} \mid y_{1:t-1}), \tag{F3}$$

which corresponds to Zhao–Cui Eq. (22). The exact posterior recursion is

$$p(\alpha, x_{0:t} \mid y_{1:t}) \propto p(\alpha, x_{0:t-1} \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t).$$

Dividing exact target by proposal gives the unnormalized correction factor

$$\omega_f(\alpha, x_{t-1:t}) = \frac{f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t)}{\widehat{p}_t(x_t \mid \alpha, x_{t-1}, y_{1:t})}. \tag{F4}$$

This is Zhao–Cui Eq. (23). The correction is conceptually important: the TT proposal can be biased, but if its conditional density is evaluable and covers the target, importance weights can correct expectations.

14 Backward Conditional Map, Path Estimation, And Smoothing

The lower conditional map goes the other direction:

$$\widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}) = \frac{\widehat{q}_t(x_t, \alpha, x_{t-1})}{\int \widehat{q}_t(x_t, \alpha, s)\,\mathrm{d}s}. \tag{B1}$$

The corresponding inverse conditional KR map samples

$$\widehat{X}_{t-1}^{(i)} = (F_{t,t-1}^{l})^{-1}\left(\Xi_{t-1}^{(i)} \mid \widehat{X}_t^{(i)}, \widehat{\alpha}^{(i)}\right). \tag{B2}$$

This is Zhao–Cui Eq. (24).

Starting at final time T, draw

$$(\widehat{X}_T, \widehat{\alpha}, \widehat{X}_{T-1}) \sim \widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}),$$

then move backward using (B2). The resulting approximate path density factors as Zhao–Cui Eq. (25):

$$\widehat{p}(\alpha, x_{0:T} \mid y_{1:T}) = \widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} \widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t}). \tag{B3}$$

The exact smoothing posterior has the same Markov factorization form with exact conditionals:

$$p(\alpha, x_{0:T} \mid y_{1:T}) = p(x_T, \alpha, x_{T-1} \mid y_{1:T}) \prod_{t=1}^{T-1} p(x_{t-1} \mid x_t, \alpha, y_{1:t}). \tag{B4}$$

The approximation can therefore be corrected by an importance ratio. The unnormalized backward weight, Zhao–Cui Eq. (26), is

$$\omega_b(\alpha, x_{0:T}) = \frac{p(\alpha)p_\alpha(x_0)\prod_{t=1}^{T} f_\alpha(x_t \mid x_{t-1})g_\alpha(y_t \mid x_t)}{\widehat{p}_T(x_T, \alpha, x_{T-1} \mid y_{1:T})\prod_{t=1}^{T-1}\widehat{p}_t(x_{t-1} \mid x_t, \alpha, y_{1:t})}. \tag{B5}$$

The numerator is the exact unnormalized path posterior. The denominator is the proposal density generated by the backward TT maps.

15 Error Propagation: What It Proves And What It Does Not Prove

Section 4 of Zhao–Cui explains how local approximation errors propagate. The key identity is the triangle inequality. Let π_t be the exact unnormalized joint density of (x_t, α, x_{t-1}) at time t, q_t the propagated density using the previous approximation, and $\widehat{\pi}_t$ the new TT approximation. Then

$$D(\widehat{\pi}_t, \pi_t) \le D(\widehat{\pi}_t, q_t) + D(q_t, \pi_t). \tag{E1}$$

The first term is the new fit error. The second is inherited from the previous time step. Under bounded likelihood or bounded prediction-likelihood conditions, the inherited error is multiplied by a finite constant.

For squared TT, Zhao–Cui use an L^2 error on square roots because it controls Hellinger distance after normalization. The practical message is precise but modest: if every local square-root approximation is accurate enough and the model does not amplify errors too severely, the sequence of approximate densities can remain controlled. This is not a theorem that ranks will stay small, that the companion implementation is globally differentiable, or that a particular high-dimensional model will be accurate without diagnostics.

16 Preconditioning

The direct target $q_t(x_t, \alpha, x_{t-1})$ can be sharply concentrated. A TT may then need high rank. Zhao–Cui Section 5 introduces a change of variables that makes the function closer to a product reference density before fitting.

Let

$$r = (x_t, \alpha, x_{t-1}), \qquad u = T_t(r),$$

and let $\eta(u) = \eta_1(u_1) \cdots \eta_D(u_D)$ be a product reference density. Choose a bridging density $\rho_t(r)$ that is easier to approximate than $q_t(r)$, and construct T_t so that the pushforward of ρ_t is close to η:

$$(T_t)_{\#}\rho_t(u) = \rho_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right| \propto \eta(u). \tag{P1}$$

This is Zhao–Cui Eq. (30).

The pushforward of q_t is

$$(T_t)_{\#}q_t(u) = q_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right|.$$

Using (P1), the Jacobian factor is proportional to $\eta(u)/\rho_t(T_t^{-1}(u))$, so the function to fit is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\rho_t(T_t^{-1}(u))}\eta(u). \tag{P2}$$

This is Zhao–Cui Eq. (31). If ρ_t captures the main shape of q_t, then q_t/ρ_t is flatter than q_t, and TT ranks may be smaller.

Fit a square-root TT in u-space:

$$\sqrt{q_t^\sharp(u)} \approx \phi_t^\sharp(u),$$

and define

$$\widehat{\nu}_t^\sharp(u) = \phi_t^\sharp(u)^2 + \tau_t^\sharp \eta(u). \tag{P3}$$

This is Zhao–Cui Eq. (32). Pulling the approximation back to physical coordinates gives

$$\widehat{\pi}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))}\rho_t(r). \tag{P4}$$

This is Zhao–Cui Eq. (33). The normalizer is unchanged by the change of variables:

$$\int \widehat{\pi}_t(r)\,\mathrm{d}r = \int \widehat{\nu}_t^\sharp(u)\,\mathrm{d}u. \tag{P5}$$

A simple bridge is Gaussian. Estimate a mean and covariance for r, set $\rho_t = \mathcal{N}(\mu_t, \Sigma_t)$, and use an affine Gaussian whitening map. A nonlinear bridge uses powers of the three factors in q_t:

$$\rho_t(r) = \widehat{p}_{t-1}(x_{t-1}, \alpha)^{\beta_\pi} f_\alpha(x_t \mid x_{t-1})^{\beta_f} g_\alpha(y_t \mid x_t)^{\beta_g}, \qquad \beta_\pi, \beta_f, \beta_g \in [0, 1]. \tag{P6}$$

This is Zhao–Cui Eq. (34). A squared-TT approximation of this bridge is

$$\widehat{\rho}_t(r) = \phi_{\rho,t}(r)^2 + \tau_{\rho,t}\lambda(r). \tag{P7}$$

This is Zhao–Cui Eq. (35). The same algebra as (P4) applies with ρ_t replaced by $\widehat{\rho}_t$.

17 Implementable Branch Objects And Data Structures

A reader implementing a minimal method must store more than an equation. At time t, a squared-TT filter object contains:

$$\mathcal{B}_t = \{\Omega_{t,k}, \Psi_{t,k}, \psi_{t,k,\ell}, C_{t,k}, R_{t,k}, B_{t,k}, \lambda_{t,k}, \tau_t, c_t, \text{mass contractions}, \text{map data}\}.$$

Here $\Omega_{t,k}$ is the coordinate domain, $\Psi_{t,k}$ maps reference coordinates to physical coordinates, $\psi_{t,k,\ell}$ are basis functions, $C_{t,k}$ are TT cores, $R_{t,k}$ are ranks, $B_{t,k}$ are mass matrices, $\lambda_{t,k}$ are reference densities, τ_t is the defensive mass, and c_t is any log-scaling shift used during fitting.

For the adaptive Zhao–Cui algorithm these objects may change with the data, with samples, and with approximation decisions. For a differentiable same-scalar target, the realized branch must be frozen before differentiation.

17.1 What A Core Object Looks Like

For coordinate k, store

$$C_k \in \mathbb{R}^{R_{k-1} \times p_k \times R_k}.$$

Given a scalar coordinate value r_k, first compute the basis vector

$$b_k(r_k) = \big(\psi_{k,0}(r_k), \dots, \psi_{k,p_k-1}(r_k)\big)^\top.$$

Then form the matrix

$$H_k(r_k)[a,b] = \sum_{\ell=0}^{p_k-1} C_k[a,\ell,b]\, b_k(r_k)[\ell]. \tag{D1}$$

Evaluation of the TT is the loop

$$v_0 = 1, \qquad v_k = v_{k-1} H_k(r_k), \quad k = 1, \dots, D, \qquad \widehat{h}(r) = v_D. \tag{D2}$$

The square-density object additionally stores mass contractions

$$M_{>k} = \int H_{k+1}(r_{k+1}) \cdots H_D(r_D) H_D(r_D)^\top \cdots H_{k+1}(r_{k+1})^\top \, \mathrm{d}r_{k+1:D}. \tag{D3}$$

These matrices are enough to evaluate marginals from the left. Analogous left-side mass contractions evaluate reverse-order marginals.

17.2 What Must Be Saved For The Next Time Step

After time t, the next step needs to evaluate

$$\widehat{p}_t(x_t;\beta) \quad \text{and} \quad \partial_{\beta_i} \widehat{p}_t(x_t;\beta)$$

at future fitting points. It is not enough to store a cloud of samples or only $\widehat{Z}_t$. A fixed-branch derivative implementation stores:

$$\left\{ C_{t,k}, \dot{C}^{(i)}_{t,k}, M_{t,>k}, \dot{M}^{(i)}_{t,>k}, \widehat{Z}_t, \dot{\widehat{Z}}^{(i)}_t, \Psi_t, \lambda_t, \tau_t \right\}. \tag{D4}$$

The derivative of the normalized marginal has the quotient form

$$\partial_{\beta_i} \widehat{p}_t(x_t;\beta) = \frac{\partial_{\beta_i} a_t(x_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(x_t;\beta) \partial_{\beta_i} \widehat{Z}_t(\beta)}{\widehat{Z}_t(\beta)^2}, \tag{D5}$$

where

$$a_t(x_t;\beta) = \int \widehat{q}_t(x_t, x_{t-1};\beta)\,\mathrm{d}x_{t-1} \tag{D6}$$

is the unnormalized retained numerator. Equation (D5) is the carried-filter derivative needed in (G2) at the next time step.

Lemma 1 (Carried filter object feeds the next target)**.** *Fix the branch at time t. Suppose the stored object contains*

$$\mathcal{F}_t = \{a_t, \dot{a}_t^{(i)}, \widehat{Z}_t, \dot{\widehat{Z}}_t^{(i)}, \Psi_t, \lambda_t, \tau_t\}.$$

Define $\widehat{p}_t$ and $\dot{\widehat{p}}_t^{(i)}$ by (D5). Then the next transformed target and its derivative at any next-step fitting point are computable from $\mathcal{F}_t$, the model density evaluators, and their derivatives.

Proof. At time $t+1$, the fixed-branch target has the form

$$\widetilde{q}_{t+1}(z;\beta) = \widehat{p}_t(x_t(z);\beta) f_\beta(x_{t+1}(z) \mid x_t(z)) g_\beta(y_{t+1} \mid x_{t+1}(z)) J_{t+1}(z).$$

The stored object evaluates $\widehat{p}_t$. The model supplies f_β and g_β, and the branch supplies J_{t+1}. Differentiating gives the product-rule expression (G2) with t replaced by $t+1$. The first term uses $\dot{\widehat{p}}_t^{(i)}$, which is the quotient derivative in (D5). No old samples or external source-paper objects are needed. □

18 One Concrete Branch, Fully Instantiated

This section gives a single implementation lane. It is intentionally narrower than Zhao–Cui's adaptive code. Its purpose is to remove ambiguity. A coder can implement this lane first, then replace pieces with adaptive or higher performance choices later.

18.1 Coordinate Order And Domain

For the fixed-parameter likelihood case, use the two-block coordinate order

$$r_t = (x_t, x_{t-1}) \in \mathbb{R}^2$$

for a scalar state. For vector states, concatenate coordinates as

$$r_t = (x_{t,1}, \ldots, x_{t,m_x}, x_{t-1,1}, \ldots, x_{t-1,m_x}).$$

Choose a fixed interval for each coordinate,

$$r_{t,k} \in [\ell_{t,k}, u_{t,k}],$$

and map from reference coordinate $z_k \in [-1,1]$ by

$$r_{t,k} = a_{t,k} + h_{t,k} z_k, \qquad a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \qquad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}. \tag{C1}$$

The Jacobian is constant:

$$J_t = \prod_{k=1}^{D} h_{t,k}. \tag{C2}$$

The branch stores $\ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}$. In the same-scalar derivative these are constants and $\dot{J}_t = 0$.

18.2 Basis, Points, Weights, Ranks

Use normalized Legendre basis with the same degree p in every coordinate:

$$b_k(z_k) = \left(\sqrt{\frac{1}{2}} P_0(z_k), \sqrt{\frac{3}{2}} P_1(z_k), \ldots, \sqrt{\frac{2p-1}{2}} P_{p-1}(z_k) \right)^\top. \tag{C3}$$

Use fixed Halton-type fitting points. Let $p_k^\star$ be the k-th prime. If

$$j = \sum_{m=0}^{M} d_m (p_k^\star)^m, \qquad d_m \in \{0, \ldots, p_k^\star - 1\},$$

define

$$\mathrm{radinv}_{p_k^\star}(j) = \sum_{m=0}^{M} d_m (p_k^\star)^{-(m+1)}, \qquad z_k^{(j)} = 2\,\mathrm{radinv}_{p_k^\star}(j) - 1. \tag{C4}$$

Use $j = 1, \ldots, N_{\mathrm{fit}}$, weights

$$W = \frac{1}{N_{\mathrm{fit}}} I, \tag{C5}$$

and fixed ranks

$$R_0 = R_D = 1, \qquad R_1, \ldots, R_{D-1} \text{ declared before fitting.} \tag{C6}$$

No pivot, point, rank, or domain choice is allowed to change during a same-scalar derivative check.

18.3 Defensive Reference, Mass, And Shift

Use the uniform reference on $[-1, 1]^D$:

$$\lambda(z) = 2^{-D}. \tag{C7}$$

Choose a defensive floor $\epsilon_\tau > 0$ and set

$$\tau_t = 2^D \epsilon_\tau. \tag{C8}$$

Then $\tau_t \lambda(z) = \epsilon_\tau$. Choose the stabilizing shift at the base parameter β_0:

$$c_t = -\max_{1 \le j \le N_{\mathrm{fit}}} \log \widetilde{q}_t(z^{(j)}; \beta_0). \tag{C9}$$

The fitted square-root target is

$$y_j(\beta) = \exp(c_t/2) \sqrt{\widetilde{q}_t(z^{(j)}; \beta)}. \tag{C10}$$

For same-branch finite differences, c_t is reused for $\beta_0 + h$ and $\beta_0 - h$. Recomputing c_t changes the scalar.

18.4 Boxed Convention: Shifted Fit, Unshifted Density

The concrete branch uses this convention everywhere downstream.

$$\boxed{\begin{aligned}
&\text{Fitted TT:} \qquad \phi_t(z; \beta) \approx e^{c_t/2} \sqrt{\widetilde{q}_t(z; \beta)}, \\
&\text{Approximate joint density:} \qquad \widehat{q}_t(z; \beta) = e^{-c_t} \phi_t(z; \beta)^2 + \tau_t \lambda_t(z), \\
&\text{Retained numerator:} \qquad a_t(z_t; \beta) = \int \left[e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \right] \mathrm{d}z_{t-1}, \\
&\text{Normalizer:} \qquad \widehat{Z}_t(\beta) = \int \widehat{q}_t(z; \beta)\, \mathrm{d}z = e^{-c_t} \int \phi_t(z; \beta)^2\, \mathrm{d}z + \tau_t.
\end{aligned}} \tag{C11}$$

Every carried filter, proposition, derivative, and finite-difference check uses (C11). If an implementation instead absorbs $e^{-c_t/2}$ into the TT cores, then it must remove the explicit e^{-c_t} factors everywhere. Mixing the two conventions changes the scalar.

18.5 Initialization And Sweep Order

Initialize every core to zero except the constant channel:

$$C_k[1,0,1] = 1, \qquad k = 2, \dots, D,$$

and

$$C_1[1,0,1] = \bar{y}, \qquad \bar{y} = \frac{1}{N_{\text{fit}}} \sum_{j=1}^{N_{\text{fit}}} y_j.$$

If a rank is larger than one, all non-first rank channels start at zero. This plain initialization gives a constant initial approximation and avoids a random initialization branch.

One bidirectional sweep means update cores $1, 2, \dots, D$, then $D, D-1, \dots, 1$. Fix the number of bidirectional sweeps S. The core update is the ridge solve (A1-2d). The branch stores the sweep order, S, ρ, and the final cores.

18.6 Pseudocode For One Forward Step

1. Build reference fitting points $z^{(j)}$ by (C4).
2. Map them to physical points $r^{(j)} = \Psi_t(z^{(j)})$.
3. Evaluate
$$\widetilde{q}_{t,j} = \widehat{p}_{t-1}(x_{t-1}^{(j)}; \beta) f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}) g_\beta(y_t \mid x_t^{(j)}) J_t.$$
4. Form $y_j = \exp(c_t/2)\sqrt{\widetilde{q}_{t,j}}$.
5. Initialize TT cores by the constant-channel rule.
6. For each fixed sweep and each core k, form environments
$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}),$$
$$R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}),$$
then form $A_{j,k}$ by (A1-2b) and solve (A1-2d).
7. After fitting, compute square-mass contractions by (M5)–(M6).
8. Compute
$$R_t = \int \phi_t(z)^2 \, \mathrm{d}z, \qquad \widehat{Z}_t = e^{-c_t} R_t + \tau_t.$$
9. Build the carried numerator a_t by integrating old-state coordinates using the same convention:
$$a_t(z_t; \beta) = \int \left[e^{-c_t} \phi_t(z_t, z_{t-1}; \beta)^2 + \tau_t \lambda_t(z_t, z_{t-1}) \right] \mathrm{d}z_{t-1}.$$
10. Store $a_t, \widehat{Z}_t$, branch objects, cores, and mass contractions.

18.7 Conditional-Map Inversion Protocol

For a conditional coordinate r_k, compute

$$F_k(s \mid r_{<k}) = \int_{\ell_k}^{s} \widehat{p}(r'_k \mid r_{<k})\, \mathrm{d}r'_k.$$

To sample from the conditional, given $u \in (0,1)$, solve

$$F_k(s \mid r_{<k}) - u = 0, \qquad s \in [\ell_k, u_k].$$

Use bisection as the baseline because it only needs monotonicity. Fix a tolerance $\varepsilon_{\mathrm{root}}$ and maximum iterations N_{root}. The branch stores both. The bisection update is:

$$m = \frac{a+b}{2}, \qquad \text{if } F_k(m \mid r_{<k}) < u \text{ set } a = m, \text{ else set } b = m.$$

Stop when $b - a \le \varepsilon_{\mathrm{root}}$ or the iteration count reaches N_{root}. Raise a failure diagnostic if the final residual $|F_k(m \mid r_{<k}) - u|$ exceeds the declared tolerance.

19 Fixed-Branch TT Filtering Recursion

The fixed-branch BayesFilter variant chooses all discrete approximation objects before differentiating. It is narrower than the full Zhao–Cui adaptive code, but it is the variant for which an analytical derivative can be stated without changing the scalar under the derivative.

There is one important notational separation. In the Zhao–Cui learning algorithm, the unknown parameter α may be carried as a random variable inside the posterior density. In an HMC-style likelihood calculation, the parameter is instead an external argument that is varied by the sampler. To avoid using the same symbol for both roles, this section writes the external differentiated parameter as

$$\beta \in \mathbb{R}^{m_\beta}.$$

The fixed-branch likelihood variant conditions on a trial β and filters only the latent state. Its unnormalized update is

$$q_t(x_t, x_{t-1}; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t).$$

If one also wants to carry a learned static parameter as a random coordinate, that coordinate can be appended to the state; the derivative below is still with respect to the external β, not with respect to an integration variable.

Let $z \in [-1,1]^D$ be reference coordinates and

$$r = \Psi_t(z) = (x_t, x_{t-1}).$$

Let $J_t(z) = |\det \nabla \Psi_t(z)|$. The transformed target is

$$\widetilde{q}_t(z; \beta) = \widehat{p}_{t-1}(x_{t-1}; \beta) f_\beta(x_t \mid x_{t-1}) g_\beta(y_t \mid x_t) J_t(z). \tag{FB1}$$

The branch fixes points $z^{(j)}$, weights w_j, basis functions, ranks, and a core-construction algorithm. The fitted square-root TT is

$$\phi_t(z; \beta) = G_{t,1}(z_1; \beta) \cdots G_{t,D}(z_D; \beta). \tag{FB2}$$

The fixed-branch approximate density is

$$\widehat{q}_t(z;\beta) = e^{-c_t}\phi_t(z;\beta)^2 + \tau_t\lambda_t(z), \qquad \int \lambda_t(z)\,\mathrm{d}z = 1. \tag{FB3}$$

Its normalizer is

$$\widehat{Z}_t(\beta) = \int \widehat{q}_t(z;\beta)\,\mathrm{d}z = e^{-c_t}\int \phi_t(z;\beta)^2\,\mathrm{d}z + \tau_t. \tag{FB4}$$

This is exactly the boxed convention (C11). The next filter object is the normalized marginal

$$\widehat{p}_t(x_t;\beta) = \frac{\int \widehat{q}_t(z_t, z_{t-1};\beta)\,\mathrm{d}z_{t-1}}{\widehat{Z}_t(\beta)}\,\left|\det \nabla z_t/\nabla x_t\right|. \tag{FB5}$$

The Jacobian factor appears if the filter is evaluated in physical coordinates. If the whole recursion is implemented in reference coordinates, it is absorbed into the stored domain maps.

Proposition 1 (Fixed-branch recursion is a normalized approximate filter)**.** *Fix a time horizon T, model densities f_β, g_β, initial density $p_\beta(x_0)$, and fixed branch objects for each time step. Suppose every $\widehat{q}_t(\cdot;\beta)$ defined by (FB3) is nonnegative, integrable, and has $0 < \widehat{Z}_t(\beta) < \infty$. Define $\widehat{p}_t$ by (FB5), starting from the fixed initial approximation. Then each $\widehat{p}_t$ is a normalized density on the retained state variable x_t. Moreover, the recursion is exactly the Bayes filtering recursion applied to the declared approximate joint density $\widehat{q}_t$, not to the exact joint density.*

Proof. Nonnegativity follows from $\phi_t^2 \geq 0$, $\tau_t > 0$, and $\lambda_t \geq 0$. Since $\widehat{Z}_t = \int \widehat{q}_t$ is positive and finite, the normalized joint density

$$\widehat{p}_t^{\,\mathrm{joint}}(z) = \widehat{q}_t(z)/\widehat{Z}_t$$

integrates to one. The retained filter is the marginal of this normalized joint density:

$$\int \widehat{p}_t(z_t)\,\mathrm{d}z_t = \int \left[\int \widehat{p}_t^{\,\mathrm{joint}}(z_t, z_{t-1})\,\mathrm{d}z_{t-1}\right]\mathrm{d}z_t = \int \widehat{p}_t^{\,\mathrm{joint}}(z)\,\mathrm{d}z = 1.$$

The change-of-variables factor in physical coordinates is the standard Jacobian factor for density transformation, so normalization is preserved. The construction uses the exact Bayes filtering algebra with q_t replaced by the declared approximation $\widehat{q}_t$. No step asserts that $\widehat{q}_t = q_t$, so the result is a normalized approximate filtering recursion rather than an exact-posterior theorem. □

20 Same-Scalar Analytical Derivative

For parameter inference one may want the approximate log evidence

$$\widehat{\ell}_T(\beta) = \sum_{t=1}^{T} \log \widehat{Z}_t(\beta). \tag{G1}$$

The word "same" matters. The derivative must be the derivative of the scalar actually computed by the forward recursion: same domains, same points, same ranks, same defensive mass, same scaling shifts, same linear solves, and same stored filter objects.

Let a dot denote $\partial/\partial\beta_i$. At fitting point $z^{(j)}$, the transformed target derivative is

$$\dot{\widetilde{q}}_{t,j} = J_{t,j}\left[\dot{\widehat{p}}_{t-1}(x_{t-1}^{(j)};\beta) f_{\beta,j} g_{\beta,j} + \widehat{p}_{t-1,j}\dot{f}_{\beta,j} g_{\beta,j} + \widehat{p}_{t-1,j} f_{\beta,j}\dot{g}_{\beta,j}\right] + \widehat{p}_{t-1,j} f_{\beta,j} g_{\beta,j}\dot{J}_{t,j}. \tag{G2}$$

For a frozen domain map, $\dot{J}_{t,j} = 0$. If the fitted target is

$$y_{t,j} = e^{c_t/2}\sqrt{\widetilde{q}_{t,j}},$$

with frozen c_t, then

$$\dot{y}_{t,j} = \frac{e^{c_t/2}}{2\sqrt{\widetilde{q}_{t,j}}}\dot{\widetilde{q}}_{t,j}, \tag{G3}$$

where a positive floor is needed in implementation if $\widetilde{q}_{t,j}$ is near zero.

For a fixed least-squares core update, write the normal equations as

$$N_k g_k = d_k, \qquad N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{G4}$$

Here $g_k = \mathrm{vec}(C_k)$. Differentiating gives

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k, \tag{G5}$$

with

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \qquad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}. \tag{G6}$$

The design matrix derivative $\dot{A}_k$ comes from the left and right TT environments, which depend on already differentiated neighboring cores during the fixed sweep order. Thus the derivative follows the same ordered sequence of stored linear solves as the forward core construction.

20.1 Differentiated Environments In Indices

For one fitting point $z^{(j)}$, define

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \qquad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}).$$

The derivative of the left environment is propagated by

$$\dot{L}_{j,1} = 0,$$

$$\dot{L}_{j,k+1} = \dot{L}_{j,k} H_k(z_k^{(j)}) + L_{j,k} \dot{H}_k(z_k^{(j)}). \tag{G6a}$$

The derivative of the right environment is propagated by

$$\dot{R}_{j,D} = 0,$$

$$\dot{R}_{j,k-1} = \dot{H}_k(z_k^{(j)}) R_{j,k} + H_k(z_k^{(j)}) \dot{R}_{j,k}. \tag{G6b}$$

If the basis and points are frozen, then

$$\dot{H}_k(z_k^{(j)})[a,b] = \sum_{\ell=0}^{p_k-1} \dot{C}_k[a,\ell,b]\, b_k(z_k^{(j)})[\ell]. \tag{G6c}$$

Thus the design-row derivative is not mysterious. Since

$$A_{j,k} = R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k},$$

and b_k is fixed on the branch,

$$\dot{A}_{j,k} = \dot{R}_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes L_{j,k} + R_{j,k}^\top \otimes b_k(z_k^{(j)})^\top \otimes \dot{L}_{j,k}. \tag{G6d}$$

Equations (G6a)–(G6d) are the operational recipe for forming $\dot{A}_k$ in (G6).

The normalizer derivative follows from the mass contraction. If

$$R_t(\beta) = \int \phi_t(z;\beta)^2 \, \mathrm{d}z, \qquad \widehat{Z}_t(\beta) = e^{-c_t} R_t(\beta) + \tau_t,$$

and c_t, τ_t are frozen, then

$$\dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t = 2e^{-c_t} \int \phi_t(z;\beta) \dot{\phi}_t(z;\beta) \, \mathrm{d}z. \tag{G7}$$

In core form this is computed by differentiating the same square-core mass contractions used to compute R_t.

20.2 Differentiated Mass Contraction In Indices

The right mass recursion was

$$M_{>j-1}[a,a'] = \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'].$$

On a frozen branch B_j is fixed. Differentiating gives

$$\begin{aligned} \dot{M}_{>j-1}[a,a'] = &\sum_{b,b',\ell,\ell'} \dot{C}_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] \dot{M}_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] \dot{C}_j[a',\ell',b'] B_j[\ell,\ell']. \end{aligned} \tag{G7a}$$

Starting from $\dot{M}_{>D} = 0$, this recursion produces the derivative of the complete square contraction. With the shifted-target convention,

$$\widehat{Z}_t = e^{-c_t} R_t + \tau_t, \qquad \dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t, \tag{G7b}$$

because c_t and τ_t are frozen branch constants.

Therefore

$$\partial_i \widehat{\ell}_T(\beta) = \sum_{t=1}^{T} \frac{\partial_i \widehat{Z}_t(\beta)}{\widehat{Z}_t(\beta)}. \tag{G8}$$

Proposition 2 (Fixed-branch gradient differentiates the declared scalar)**.** *Assume all branch objects are fixed; all model densities, basis evaluations, and domain maps used on the branch are differentiable in β; every fitted core is obtained by a fixed finite sequence of ridge-regularized linear solves with nonsingular matrices N_k; and $0 < \widehat{Z}_t(\beta) < \infty$. Then the recursion (G2)–(G8) computes $\nabla_\beta \widehat{\ell}_T(\beta)$ for the scalar in (G1), where $\widehat{Z}_t$ is the normalizer computed by the fixed-branch forward pass.*

Proof. The proof is induction over time and over the fixed core-update sequence. At time 0, the initial filter evaluator and its derivative are assumed fixed and differentiable. Suppose the previous filter evaluator and derivative are available. Then the transformed target derivative (G2) follows

from the product rule and the chain rule. The square-root target derivative (G3) follows from differentiating $e^{c_t/2}\sqrt{\widetilde{q}}$, with $\dot{c}_t = 0$ on the fixed branch.

For each core update, the forward pass solves $N_k g_k = d_k$. Because N_k is nonsingular and differentiable, differentiating $N_k g_k = d_k$ yields

$$\dot{N}_k g_k + N_k \dot{g}_k = \dot{d}_k,$$

and hence (G5). The fixed sweep order composes finitely many differentiable maps, so all fitted cores are differentiable functions of β on the branch.

The square contraction $R_t = \int \phi_t^2$ is a finite contraction of differentiable core coefficients and fixed mass matrices. Differentiating that contraction and then applying the frozen scaling convention $\widehat{Z}_t = e^{-c_t} R_t + \tau_t$ gives (G7). Since $\dot{c}_t = 0$ and $\dot{\tau}_t = 0$, no additional terms appear. Finally, differentiating $\widehat{\ell}_T = \sum_t \log \widehat{Z}_t$ gives (G8). Each term is computed from the same stored objects used by the forward pass, so the derivative is of the declared scalar, not of a nearby adaptive algorithm. □

21 Numerical Stabilization, Failure Diagnostics, And Finite-Difference Test

A minimal implementation should record the following diagnostics at every time:

$$\widehat{Z}_t, \qquad \frac{\tau_t}{\widehat{Z}_t}, \qquad \max_k R_{t,k}, \qquad \kappa(N_k), \qquad \frac{\|Ag - y\|_W}{\|y\|_W}, \qquad \min_j \widetilde{q}_{t,j}.$$

The defensive mass ratio $\tau_t / \widehat{Z}_t$ detects whether the floor is dominating. The rank and condition numbers detect whether the TT fit is too hard for the chosen branch. The residual detects local fitting failure. The minimum target value detects square-root instability.

For the gradient, use same-branch finite differences. Choose a base β_0, build and freeze the branch, then evaluate

$$\frac{\widehat{\ell}_T(\beta_0 + he_i; B) - \widehat{\ell}_T(\beta_0 - he_i; B)}{2h}$$

with exactly the same branch B. Compare it to the analytical derivative from (G8). Rebuilding the branch at $\beta_0 \pm he_i$ tests a different adaptive scalar and is not a valid same-scalar derivative test.

22 Minimal Runnable Example Specification

A minimal example should use a one-dimensional nonlinear state model with one unknown scalar parameter:

$$x_t = \beta x_{t-1} + \sigma \epsilon_t, \qquad y_t = \sin(x_t) + \eta_t,$$

with Gaussian noises. Use $T = 2$ so the second step must evaluate the carried filter from step 1. Use a fixed reference interval, fixed Legendre basis, fixed fitting points, fixed rank one or two, fixed ridge, fixed τ_t, and fixed scaling shifts. The script should print:

$$\widehat{\ell}_T(\beta_0), \qquad \partial_\beta \widehat{\ell}_T(\beta_0), \qquad \text{central finite-difference error.}$$

Passing this example does not prove high-dimensional performance. It proves that the value path and derivative path are aligned for the declared scalar.

22.1 Two-Step Value Path

The two-step example should be written so the second step cannot cheat. At time 1, build

$$q_1(x_1, x_0; \beta) = p_\beta(x_0) f_\beta(x_1 \mid x_0) g_\beta(y_1 \mid x_1).$$

Fit ϕ_1, form

$$\widehat{q}_1(z_1, z_0; \beta) = e^{-c_1} \phi_1(z_1, z_0; \beta)^2 + \tau_1 \lambda_1(z_1, z_0),$$

and compute

$$\widehat{Z}_1(\beta) = e^{-c_1} \int \phi_1(z_1, z_0; \beta)^2 \, \mathrm{d}z_1 \, \mathrm{d}z_0 + \tau_1.$$

Then store the carried filter

$$\widehat{p}_1(x_1; \beta) = \frac{\int \widehat{q}_1(z_1, z_0; \beta) \, \mathrm{d}z_0}{\widehat{Z}_1(\beta)} \left| \frac{\mathrm{d}z_1}{\mathrm{d}x_1} \right|.$$

At time 2, the target must call this stored evaluator:

$$q_2(x_2, x_1; \beta) = \widehat{p}_1(x_1; \beta) f_\beta(x_2 \mid x_1) g_\beta(y_2 \mid x_2).$$

The total scalar is

$$\widehat{\ell}_2(\beta) = \log \widehat{Z}_1(\beta) + \log \widehat{Z}_2(\beta).$$

Here $\widehat{Z}_2$ is computed by the same convention after fitting ϕ_2 to the shifted square-root target for q_2:

$$\widehat{q}_2(z_2, z_1; \beta) = e^{-c_2} \phi_2(z_2, z_1; \beta)^2 + \tau_2 \lambda_2(z_2, z_1), \qquad \widehat{Z}_2(\beta) = e^{-c_2} \int \phi_2(z_2, z_1; \beta)^2 \, \mathrm{d}z_2 \, \mathrm{d}z_1 + \tau_2.$$

The derivative path must carry $\partial_\beta \widehat{p}_1(x_1; \beta)$ into the derivative of q_2. If a script recomputes step 1 differently when evaluating step 2, it is no longer testing the fixed-branch recursion.

23 Reader-Facing Conclusion

Zhao–Cui's method is persuasive because it attacks the filtering recursion as a sequence of function-approximation problems. The posterior update creates a three-block unnormalized density in (x_t, α, x_{t-1}). A tensor train tries to store that function without a full grid. Squaring a TT for the square-root target makes the approximation nonnegative. Mass contractions give normalizers and marginals. Marginal ratios give conditional densities. Those conditional densities give KR maps for forward filtering proposals and backward smoothing proposals.

For BayesFilter's gradient-driven use case, the adaptive research algorithm must be separated from a fixed-branch scalar. Once the branch is fixed, the approximate normalizers define a clear scalar $\widehat{\ell}_T = \sum_t \log \widehat{Z}_t$, and ordinary calculus differentiates that scalar through target evaluations, TT core solves, contractions, and carried filters. The result is not a proof of exact posterior accuracy. It is a mathematically honest route from Zhao–Cui's sequential TT idea to an implementable approximate likelihood and same-scalar derivative.

A Equation And Algorithm Disposition Summary

The detailed ledger beside this note gives every numbered equation and algorithm disposition. In the main text, Zhao–Cui Eqs. (1)–(26) and (30)–(35), and Algorithms 1–5, are either reconstructed directly or discussed as support for the reconstruction. Numerical example equations from Section 6 are used only as evidence of demonstrated examples, not as theorem support.